

1. UML-to-Code Trace

UML element	Python element	Implemented ?	Notes
Patient class	<code>class Patient</code>	Yes	Stores patient details.
Patient identifier and name	<code>_patient_id,</code> <code>_name</code>	Yes	Validated when created.
Update patient details	<code>update_details(</code> <code>)</code>	Yes	Validates the new name before saving it.
Practitioner class	<code>class</code> <code>Practitioner</code>	Yes	Stores identifier, name and specialty.
Practitioner specialty	<code>_specialty,</code> <code>get_specialty()</code>	Yes	Must be nonblank text.
Appointment class	<code>class</code> <code>Appointment</code>	Yes	Stores appointment details and controls status changes.

Appointment–Patient relationship	<code>_patient</code>	Yes	References one existing Patient object.
Appointment–Practitioner relationship	<code>_practitioner</code>	Yes	References one existing Practitioner object.
Appointment date and time	<code>_scheduled_at</code>	Yes	Must be a <code>datetime</code> object.
Appointment status	<code>_status,</code> <code>AppointmentStatus</code>	Yes	Uses SCHEDULED, CANCELLED and COMPLETED.
Cancel appointment	<code>cancel()</code>	Yes	Changes a scheduled appointment to CANCELLED.
Complete appointment	<code>complete()</code>	Yes	Changes a scheduled appointment to COMPLETED.
Reschedule appointment	<code>reschedule()</code>	Yes	Only scheduled appointments can move.

2. Domain Invariants

Class	Invariant / rule	How protected
Patient	Patient ID must be nonblank text.	Constructor calls <code>required_text()</code> .
Patient	Name must be nonblank and contain at least one letter.	Constructor and update method call <code>required_name()</code> .
Patient	Contact details must be text; blank is allowed.	Checks the type before saving.
Practitioner	Identifier and specialty must be nonblank text.	Uses <code>required_text()</code> .
Practitioner	Name must contain at least one letter.	Uses <code>required_name()</code> .
Appointment	Patient and practitioner must be the correct objects.	Constructor checks types and raises <code>TypeError</code> for invalid objects.
Appointment	Date and time must be a <code>datetime</code> .	Constructor and <code>reschedule()</code> check the type.
Appointment	New appointments start as SCHEDULED.	Constructor sets the initial status.

Appointment	Cancelled and completed appointments cannot change status.	<code>change_status()</code> raises <code>InvalidStatusTransitionError</code> .
Appointment	<code>SCHEDULED</code> → <code>SCHEDULED</code> is rejected.	<code>change_status()</code> checks for the repeated status.
Appointment	Only scheduled appointments can be rescheduled.	Status is checked before changing the time.

3. Composition / Inheritance Decisions

Relationship	Decision	Rationale
Appointment and Patient	Association	Appointment references a Patient who exists independently.
Appointment and Practitioner	Association	Appointment references a Practitioner who exists independently.
Patient and Practitioner	No inheritance	They have different roles; neither is a type of the other.

Appointment and PatientRecord	Reject inheritance	An Appointment is not a type of PatientRecord.
-------------------------------	--------------------	--

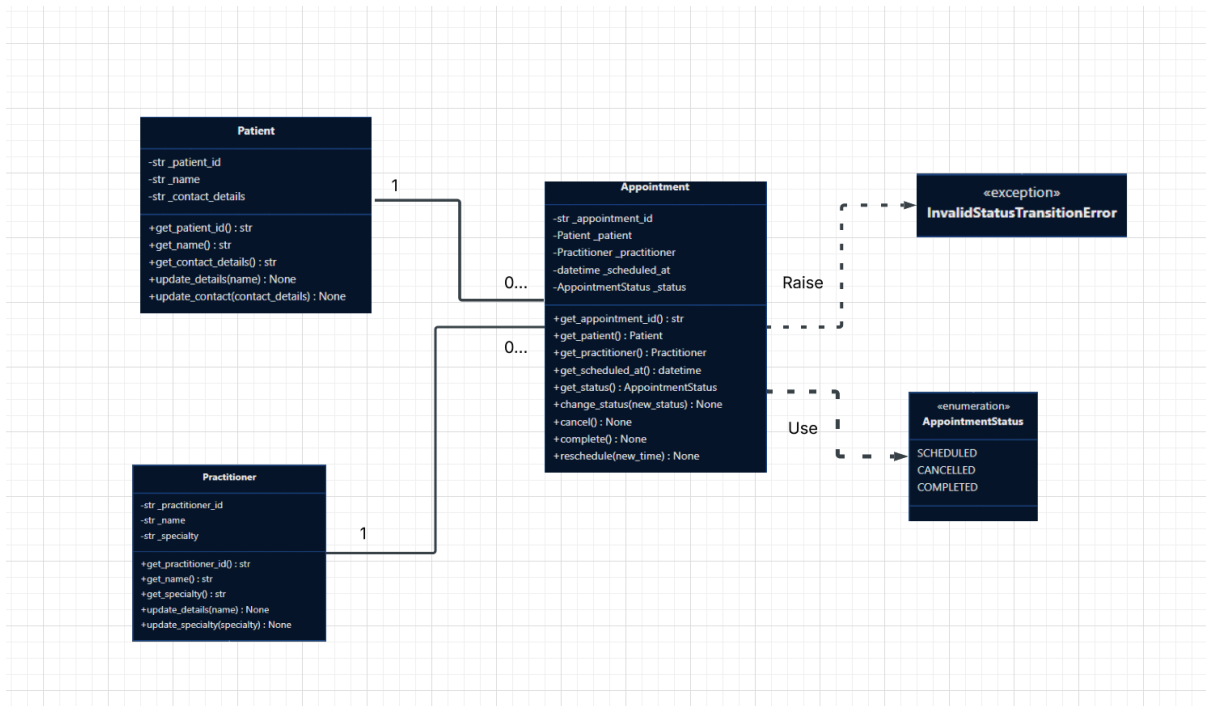
4. AI Pair-Programming Record

AI contribution	Conforms ?	Decision	Reason	Verification
Public Appointment attributes	No	Modify	Use internal <code>_</code> names and simple getters to support encapsulation.	Check getters return the stored values.
Missing Patient and Practitioner type checks	No	Modify	Prevent invalid objects being stored.	Pass strings instead of objects; expect <code>TypeError</code> .
Rescheduling cancelled or completed appointments	No	Modify	Only scheduled appointments should move.	Attempt both; expect an exception and unchanged time.
Allowing SCHEDULED → SCHEDULED	No	Modify	Reject repeated status transitions under the agreed rule.	Attempt the transition; expect <code>InvalidStatusTransitionError</code> .

<code>send_reminder()</code> method	No	Remove	Notification features are outside the lab scope.	Confirm the method is absent.
AppointmentStatus enum	Yes	Keep	Restricts status values to the agreed choices.	Create an appointment; check its initial status.
InvalidStatusTransitionError	Yes	Keep	Clearly reports illegal appointment actions.	Cancel twice; expect the exception on the second attempt.
Terminal menu and input functions	Outside Part D scope	Keep as demonstration code	Useful for manual checks, but separate from domain responsibilities.	Run the menu and check invalid input asks for a retry.

5. Updated UML

Insert updated UML only if implementation revealed a justified design change. Explain every change.



The implementation preserves the three domain classes: Patient, Practitioner and Appointment. One patient and one practitioner are still connected to the appointment. There was no new composition. Simple getter methods allow access to the values of internal attributes, which are denoted by `_` names. The `send_reminder()` function was eliminated since it was not supported.